

Cloud Practitioner - FreeCodeCamp

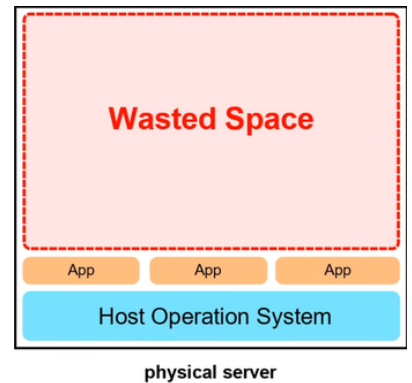
Cloud computing - ใช้ server คนอื่นประมวลผล/เก็บข้อมูลแทนที่จะใช้ของตัวเอง(On-premise)

Common cloud services for IaaS - Compute(EC2), networking(VPS), storage(EBS), databases(RDS)

Computing คลายแบบ

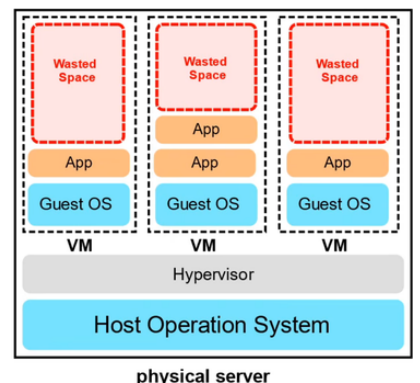
1. Dedicated (Bare Metal)

- **physical server** มาอยู่ที่ตัวเราเลย
- ขาดการแยกกัน **ไม่มี isolation ระหว่าง Application**
- **Vertical scale** (การ upgrade เครื่อง เช่น RAM, CPU) ยาก
- หลาย app อาจตีกันเรื่องแบ่งทรัพยากร (มันต้องจองแย่งกัน)
- และ environment อาจไม่ตรงกัน (dependency hell)
- อาจใช้ใน High performance computing (**พวก ต้องการ OVERHEAD ต่ำๆ**)



2. VMs (machine in machine)

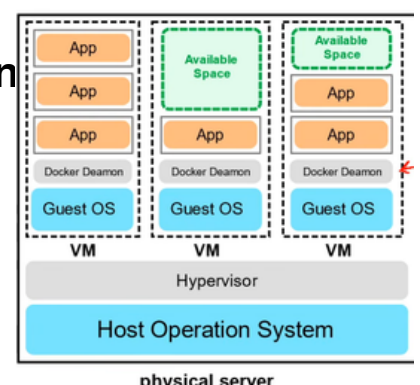
- จะ **generate new OS (guest OS)** ขึ้นมาซ้อนเป็นเหมือนอีกเครื่อง โดยมี **Hypervisor** จัดการพวก **instances VMs** (เช่น VirtualBox or AWS Nitro System)
- **Isolate** ได้ระดับนึง (different instance of VM) แต่ใน **VM** นึง **ยังรับ app พร้อมกันไม่ได้** ขนกันปัญหาเดิม
- **Vertical Scale + Horizontal Scale (เพิ่มเครื่อง)** ง่าย
- ก็แค่ไปปรับ config VM
- **ยังเสียทรัพยากรบางส่วนสูญเปล่าใน VM** อยู่
- **เพิ่มเติม overhead ที่เสียไปในการสร้าง VM**



3. Containers

ใน **OS เดียวกัน** เราต้องการให้ **run many app** โดยไม่ชน
เราก็เลยมีสิ่งที่เรียกว่า **containers** ในภาพเป็น **Docker Deamon**
ตัวจัดการ container

- container utilize mem ได้ดีมาก (**Shared Kernel**) เพราะสร้าง instance เพิ่มได้มาใช้
 - เร็ว เพราะ **ไม่ต้องไป load kernel** ใหม่แบบ VM
 - แต่ละ instance container มี dependency ต่างกัน (**complete ISOLATION**) migration ง่ายขึ้นด้วย มีข้อเสียแค่ maintain เยอะ
- ## 4. Functions (Serverless computing ex. AWS LAMBDA)
- **เราแค่โค้ดขึ้นไปรันเลยบอก environment ให้เรียบร้อย ไม่ต้องไปนั่ง maintain** แบบ container
 - **จ่ายแค่ช่วงที่รัน เลือกความจำได้** ค่อนข้างง่ายและสะดวก (**event-driven**)
 - อาจมีปัญหา **Cold-Start** ตอนรอ VM run หรือ request ไปแล้วอาจช้า + **TIME LIMIT**



Type of cloud computing

1. Software as a Service (SaaS) - ให้ software มาใช้เลย

ex. github

2. Platform as a Service (PaaS) - ให้ platform เราใช้แค่การ develop app ได้เลย

ex. AWS Elastic beanstalk , Heroku

3. Infrastructure as a Service (IaaS) - ให้มาเชิ้ระดับ OS ขึ้นไป แต่ไม่ต้องไปยุ่ง HW

ex. AWS Elastic Compute cloud(EC2)

Cloud computing deployment models

Public cloud - ใช้ CSP Shared (Multi-tenant) กับหลายเจ้า คั่นโดยกำแพงนิ่ดเดียว

Private cloud - ใช้ on premise or อาจไปขอใช้ของ CSP แต่ server นี้ต้องไม่ public

Hybrid - ผสมสอง และเชื่อมกัน

Cross cloud - ใช้ Multiple CSPs

Computing power - ภาพรวมความสามารถของระบบคอมพิวเตอร์ในการจัดการงานให้เสร็จ

ตัวที่ใช้ concept นี้ก็ EC2(General Computing) , AWS Inferentia (Inf1) ตัวนี้ใช้ใน

ML/AI , AWS Braket (Quantum Computing) อาจมี AWS graviton (ARM)

การเลือก Region เป็นสิ่งที่สำคัญเพราะ

1. ต้องถูกตามกฎหมายโซนนั้นๆ (ต้องระวังพวกข้อบังคับ)

2. cost ไม่เท่า

3. มี service ไรบ้าง

4. ระยะ / latency ไปหา end user

AWS ก็จะมี regional services และ global services ความต่างคือ global เป็น

services ที่มีอยู่หลาย region อยู่แล้วเช่น S3, cloudfront, Route53, IAM

Availability Zones (AZ) - จุดที่มี data center อาจมี 1 หรือหลายที่ ใน AZ นึง (ใน

region 1 data center จะเชื่อมกันด้วย low latency มากๆ < 10ms) สิ่งนี้ก็คือสิ่งที่บริษัท

จะรันหลายๆตัวเพื่อกันมันล่มอะนะ ถ้า common ก็คือ run บน 3 AZs (มันคือเลขตัวท้ายในตัว

region) EC2 ต้องเกิดใน Subnet และ Subnet จะต้องถูกผูกติดกับ AZ ไ่ AZ หนึ่งเสมอ

Fault tolerance terminologies

Fault domain - zone network ที่ถ้ามีตัวสำคัญระเบิด ผลกระทบก็จะอยู่แค่วงนั้น ฟลถ้า

data center ระเบิดก็ระเบิดแค่ส่วนนั้น

Fault level - รวม fault domain ซึ่งใน fault domain ก็แบ่งเป็นหลาย level อีกตาม

ระดับความรุนแรง

Region = fault level AZ = fault domain

SPOF (Single Point of Failure) - จุดที่พังแล้ว ระเบิดเลย

AWS global network - เชื่อมกันระหว่าง AWS global infrastructure

On ramps (User → AWS) - AWS global Accelerator , AWS S3 Transfer

Acceleration

Off ramps (AWS → User) - Amazon CloudFront

VPC endpoints - รับประกันว่าข้อมูลไม่รั่วสู่ public internet

Point of Presence (PoP) - จุดค้นกลางระหว่าง Users and AWS region มักจะเป็น data centers or hardware

Edge Locations - เป็น datacenter ที่ถือ cache เอาไว้ เพื่อลดระยะทาง end users อารมณ์ proxy server

Regional Edge Locations - ฝ่า L2 cache cache ที่ใหญ่กว่าตัว edge และแลกเปลี่ยนข้อมูลกับ edge locations เพื่อไม่ให้ต้องไปเอาจาก server จริง

PoP จะอยู่ในจุดนัดพบ (IXP) ที่เครือข่ายของ AWS มาเชื่อมต่อกับเครือข่ายอินเทอร์เน็ตสาธารณะ

Tier 1 network - เป็น network ที่ไปหาทุกตัวใน internet ได้ โดยไม่เสียตัง (ฝ่าลูกศรชี้ไปหา NP-Hard/Complete)

ซึ่ง PoP ก็เชื่อมตัวนี้แหละ Tier 1 network ที่ให้บริการโดย Tier 1 transit providers เพื่อส่งข้อมูล

Services ที่ใช้ PoP:

Amazon CloudFront - จะทำให้ web เราไปใช้บริการ nearest edge locations เลือกได้ว่าจะใส่ไรไปในข้อมูลที่จะให้ cache

Amazon S3 Transfer acceleration - เอา URL ที่พิเศษให้ User upload ลง edge locations ได้ ทำให้ ข้อมูลไปถึง S3 ไวขึ้น (download/upload)

AWS global accelerator - หาเส้นทางซึ่งที่สุดของ end users → web server ตัวมันอยู่ ทำให้ user ผ่าน edge location เพื่อให้เร็วมากๆ เพื่อไป web server ตัวจริง (query/update/API calls)

AWS direct connects - เราเอา ของเราไปเชื่อมกับ AWS ก็คือจะได้ option ในการเลือก ก็คือ 1.Dedicated เหมာ Port เองเลย (1G, 10G, 100G) ยึดช่อง

กับ 2.Hosted: แบ่งเช่าผ่าน Partner เริ่มต้นเบาๆ ได้ (50M - 10G) หารช่อง

Local Zones - data center ท่ามกลางคนหมู่มากมีเพื่อผู้ที่ต้องการ super low latency

AWS Wavelength Zones - edge computings ที่อยู่บน 5G network(Mobile) เหมือนเอา VM ไปเปิดที่ขอบเสาสัญญาณเพื่อ super low latency

Data Residency - ที่อยู่ข้อมูล ที่ต้องทำตาม Compliance Boundaries และ Data Sovereignty พวกกฎหมายทั้งหลาย

AWS config - เป็นตัวไว้ตรวจการตั้งค่า ซึ่งตั้งกฎได้ว่าจะไรเปลี่ยนแล้วแจ้งเตือน อาจ alert ไม่ก็ auto rollback

AWS outposts - เอา HW ไปตั้งที่เราเลย แล้วก็เชื่อมกับเขา

IAM policies - จะเลือกให้ไปหาใครหรือไม่ไปหาใคร หรือแบบคนเข้าก็ใช้ตัวนี้

AWS ground station - ใช้การสื่อสารดาวเทียมในการ ส่งสาร / ภาพถ่ายอะไรตามแต่

Cloud Architecture Terminologies

Solution Architect - นักออกแบบ การแก้ปัญหา technical โดยใช้หลายระบบ

Cloud Architect - ต่างที่โฟกัสโดยจะแก้ปัญหาโดยใช้ cloud

ข้อต้องพิจารณาเวลาออกแบบ Solution

1.Availability - เชฟควรจะรันตลอดเวลา

requirement - NO SPOF และมีหลาย AZ รันอยู่ อาจมี ELASTIC LB มาช่วยกระจาย load ได้

2.Scalability - ควรจะสามารถขยายต่อไปได้เรื่อยๆ แบบรวดเร็ว

วัดจากความสามารถในการทำ vertical + horizontal scaling

3.Elasticity - ความยืดหยุ่น solution

วัดจากการยืดหด server รับตามload เช่น Horizontal scale out/in ตัวที่ใช้ได้ คือ ASG auto scaling groups

4.Fault tolerance - กันระเบิดแค่ไหน

NO SPOF และถ้าระเบิดก็ไปรัน backup(shift traffic) สามารถใช้ RDS Multi-AZ ได้

5.Disaster recovery - ระเบิดแล้วฟื้นตัวได้แค่ไหน

ก็พวกมี backup ถึง backup เร็วแค่ไหนไร้นั้น ใช้ AWS Elastic Disaster Recovery(DRS)

และ อีก 2 ตัวเพิ่มเติม

1.Security

2.Cost

Business Continuity plan (BCP)- จะให้ทำงานยังไงตอนระเบิด

Recovery point objective (RPO) - อยากให้เสีย data มากสุดกี่นาที/ชั่วโมง

Recovery time objective (RTO) - อยากให้server ล่มกี่นาที/ชั่วโมง

สองอย่างนี้ trade off กัน

Disaster recovery choices (sort by Cost)

1.Backup and restore - backup แล้ว upload ขึ้นไปใหม่ ในตัวเดิม

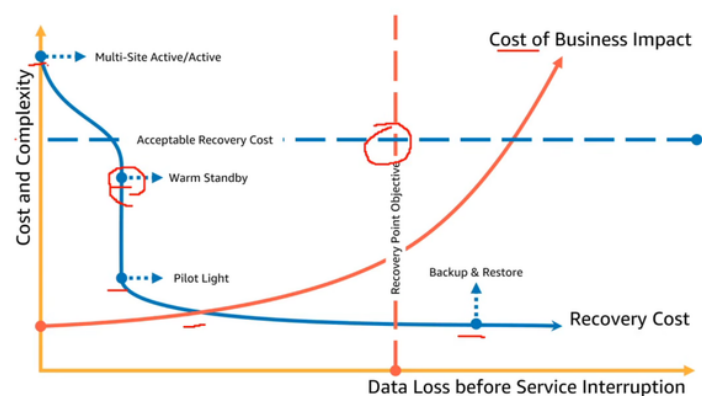
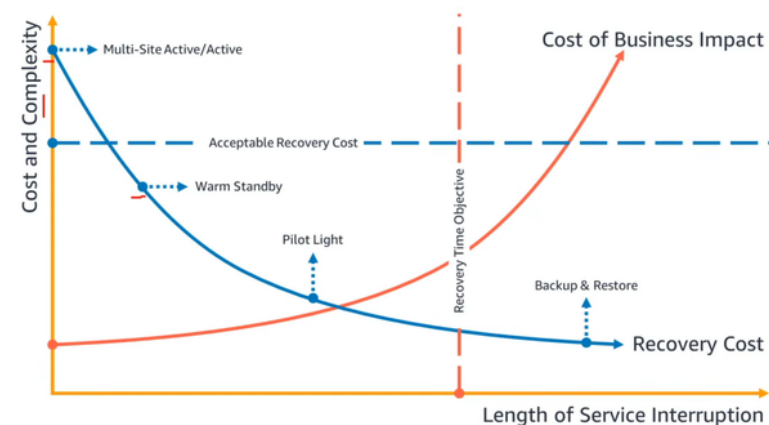
2.Pilot Light - sync DB แล้วไป run EC2 อีกโซน เปิดใหม่

3.Warm standby - sync DB มีหลาย server พังไปรันในอีกตัวที่ เล็กกว่าเปิดอยู่แล้ว

4.Multi-Site Active/active -sync DB เอาของก็อปไว้แล้ว ขึ้นไปในอีก โซนเลย ขนาดเท่า

กันเปิดอยู่แล้ว RTO

RPO



Application Programming Interface (API)- คือกฎที่ใช้บอกว่าเวลาจะทำอะไรกับฝั่ง server ที่มี API client จะต้องส่ง header + payload อะไรบ้าง (ฟ้a parameter function)

HTTP/HTTPS - เป็น protocol แ่ควบคุมวิธีการส่งหรือทำอะไรกับ อีกฝั่งเช่น ลบ ข้อมูล
AWS API - API ของ AWS

POSTMAN - ตัวช่วยเขียน FORMAT ในการใช้ API ของตัวๆนึ่ง จะมี parameter มาให้ และ ก็ไป config HTTP นิดหน่อยก็จะดึงมาได้และ

ปกติจะจัดการกับ API ด้วย AWS management console(ใช้interface web), AWS SDK(ใช้ภาษาโปรแกรม) , AWS CLI(ใช้พวกคำสั่ง shell)

Powershell - เป็น command line shell ที่ต่างจากตัวอื่นตรงที่ มันจะ return object .NET ซึ่งสามารถใช้ในการจัดการ AWS API ได้

Amazon Resource Names (ARNs) - เอาไว้ใช้ระบุว่าจะใช้อะไรของ AWS format มันคือ arn:partition:service:region:account-id:resource-type/resource_id

partition - ตามความเฉพาะของ top of region อารมณที่จีนแยกออกตัวหลักอะ services - ใช้service อะไร ex EC2 ,S3 ,IAM

Region - ใช้ AWS region ไหน

Account-id ของเรา

Resource_id - อาจเป็นเลขหรือ path

*ใช้ในการเอาทั้งหมดแบบไม่เลือกเฉพาะกลุ่มใดกลุ่มนึ่งไรจั้น ส่วนใหญ่ก็อปแปะ

::: อาจมีคั่นกลางกรณี global no region + account

.ใช้ gitpod กับ cloud9 ในการใช้พวก sdk ได้

Infrastructure as a code (IaC) - เขียน script ในการจัดการ automate ของ cloud infrastructure

AWS Cloudformation (CFN) - การเขียนแบบ explicit(ทุกอย่างต้องบอกอย่าง ละเอียด)ex.JSON,YAML

AWS Cloud Development kit (CDK) - การเขียนแบบ implicit(ใช้ของสำเร็จรูปไม่ได้ บอกตรงๆ) Python,JS,TS,C#,GO

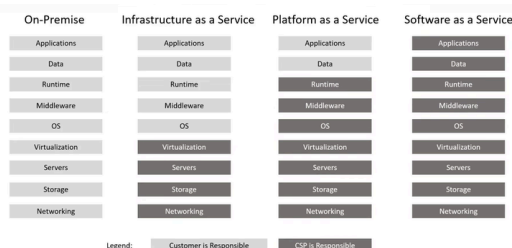
CDK ครอบ CFN ก็คือ CDK ถูกแปลงลง CFN อีกที

CDK - จัดการ Infrastructure SDK - ใช้งาน infra ที่ CDK สร้าง

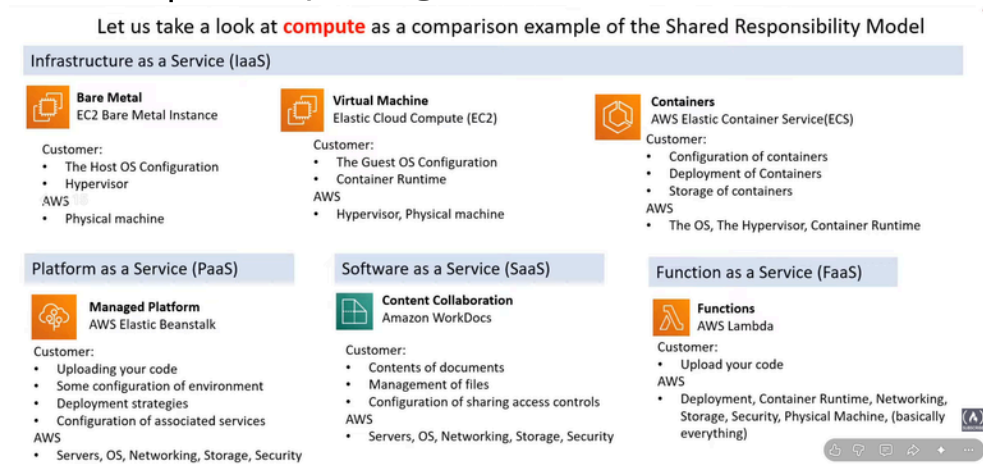
ใช้ AWS toolkit ผูกกับ VS ได้

Shared responsibility model - จะบอกประมาณว่า CSP รับผิดชอบส่วนไหน เราจะต้อง รับผิดชอบส่วนไหน แบ่งกันรับผิดชอบ มองง่ายๆคือ IN / OF the Cloud

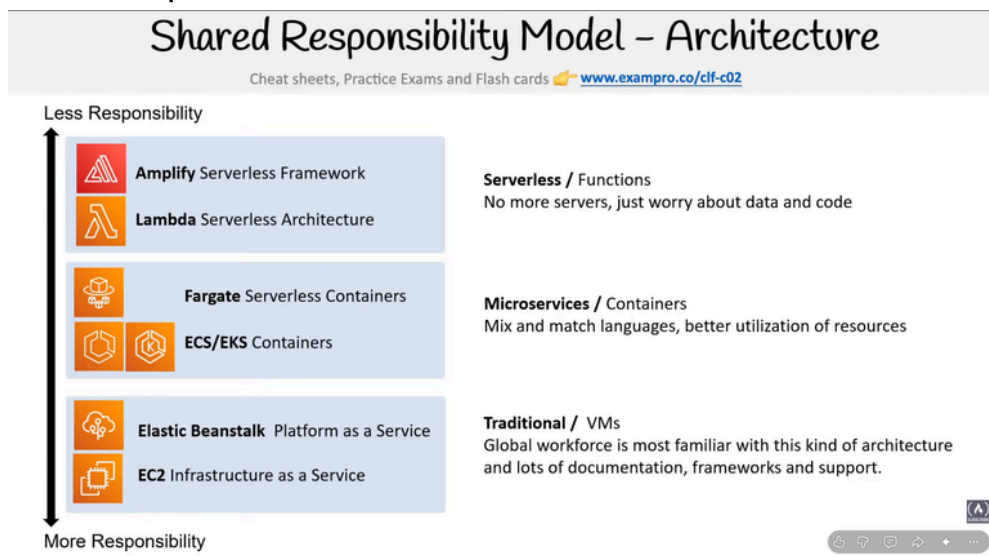
ในความต่างความรับผิดชอบในความต่างของ service คืออันที่เราต้องต้อง set นั้นแหละ



SRM ในยุ computing ex.



SRM ในยุ architecture ex



EC2 - สร้าง VM ได้ เรียกเป็น instance จัดการ os,cpu,ram,networks มีศัพท์เพิ่มเติม AMI (amazon machine image) - VM ที่ preconfig AWS services อื่นมันก็ on top EC2 เป็น backbone

Amazon Lightsail - EC2 but easier

Elastic Container Services (ECS) - EC2 instance but with docker

Elastic Container Registry(ECR) - เก็บ container images (อะไรที่แอปเราต้องใช้ในการรัน)

ECS Fargate - เป็น serverless ที่ง่ายตาม container ที่รัน ไม่มีไม่จ่ายถ้า ECS ต้องจ่ายค่า EC2 ถึงไม่ได้เปิด container

Elastic Kubernetes Services(EKS) - เอาไว้ใช้ K8s

AWS Lambda - serverless ส่งcode config mem function รันค้างเท่าไร ก็จะโดนตาม ที่มัน trigger เท่านั้น + 15 mins limit TLE

Higher Performance Computing services(HPCs) -HPC คือ เอาคอมหลายๆตัวมาช่วยคำนวณก็ supercomputer

The Nitro System - เน้นทำให้เร็วและปลอดภัย

มี Bottlerocket ที่ไว้ใช้รันLinux ที่เข้าใกล้ bare metal ได้

HPC - มี AWS Parallelcluster

Edge and Hybrid Computing services

Edge computing - เอา computing workload ไปอยู่ขอบใกล้ตัวเป้าหมายให้มากที่สุด

Hybrid Computing - เอา workload รันทั้ง datacenter เราและ cloud ของ CSP
ก็พวก AWS outpost (physical rack), AWS Wavelength (เอาไปตั้งใน 5G), VMWare
Cloud on AWS (ยก VMWare เราไปใส่ bare metal เขา), AWS local zones (ไปใช้
edge datacenters)

Cost and Capacity Management Computing Services

ต้องการลด Cost และ Capacity ต้องรองรับ traffic usage ได้

EC2 Spot Instances - แบบเช่าชั่วคราว ที่พร้อมโดนเจ้าของเตะออกเสมอถ้ามีแขก VIP มา
ขอจ่ายราคาเต็ม

AWS Batch - ใช้วางแผนคำนวณ workloads

AWS Compute Optimizer - ลด costs และเพิ่ม performance โดย AI

ASGs - ย่อขนาด resource ที่ใช้ตามสถานการณ์

ELB - แบ่ง load ไม่ใช่โหนดทีเดียว (ลดการ vertical scaling)

EB - ลด operational cost โดยการไม่ set เอง ... เราแค่ไป deploy